CSCE 3301 - Computer Architecture Project 1

Names: Ziad Eliwa - Mostafa Abdelwahed
Student ID: 900246124 - 900243064

Milestone: II

1 Introduction

In this project, we built a 32-bit single-cycle RISC-V processor using Verilog. The goal was to connect the main parts of a CPU and run different types of RISC-V instructions. We also wrote testbenches to check the design in simulation.

The processor handles integer arithmetic, logic, shifts, loads, stores, branches, and jumps. It also has simple behavior for a few system instructions. Each instruction is completed through one datapath cycle, without pipelining.

2 Datapath Design

The program counter (PC) sends an address to the instruction memory. The instruction is decoded by the control unit, while the register file provides the source values. An immediate generator prepares constants and offsets. The ALU then performs the needed operation, and the result is sent to a register or to data memory.

Normally, the PC moves forward by four bytes. For branches and jumps, the branch control unit selects a new target. JAL and JALR write PC+4 into the destination register. For JALR, bit 0 of the target is cleared. EBREAK uses a halt signal to keep the PC at the same address.

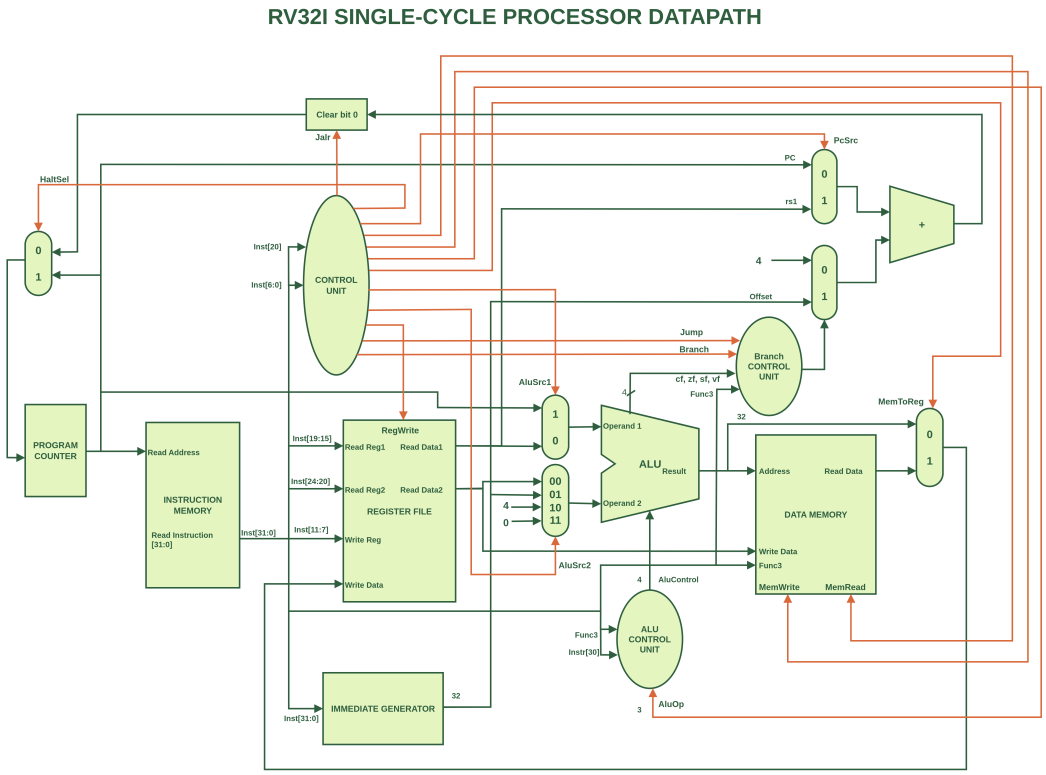


Figure 1: The single-cycle RV32I datapath, showing the PC, memories, register file, ALU, immediate generator, control units, and multiplexers. File: `RISCV Single Cycle Datapath.png`.

3 RTL Implementation

We divided the design into small modules to make it easier to build and test. The top-level `cpu.v` file connects these modules together.

RTL file(s)	Purpose
`cpu.v`	Connects the complete processor datapath.
`control_unit.v`	Creates control signals from the opcode.
`alu_control_unit.v`, `branch_cu.v`	Select the ALU operation and decide whether a branch is taken.
`prv32_ALU.v`, `shifter.v`	Handle arithmetic, logic, comparison, and shift operations.
`imm_generator.v`	Builds instruction offsets and constants.
`register_file.v`, `decoder_5_by_32.v`	Implement the 32 general-purpose registers and write selection.
`instr_mem.v`, `data_mem.v`	Store instructions and handle byte, halfword, and word memory operations.
`dff.v`, `nbit_register.v`, `mux2x1.v`, `nbit_mux2by1.v`, `nbit_mux4by1.v`	Provide reusable storage and selection circuits.
`defines.v`	Contains shared instruction and ALU definitions.

Table 1: Main RTL files and their roles.

3.1 Registers and Memory

The register file has 32 registers, and register x0 stays at zero. Register writes happen on the rising clock edge, while register reads are combinational. The PC is also updated on the rising edge.

The instruction memory contains 64 words. The data memory also contains 64 words, giving it a 256-byte address range. Data memory supports byte, halfword, and word accesses. Stores happen on the rising clock edge, while loads are read combinationally.

4 Testing Method

We used separate testbenches for the smaller modules and for the complete CPU. The shared `common_tb.vh` file creates RISC-V instruction encodings, loads a test program into instruction memory, runs the clock, and checks final register values. Each full-CPU test is run as its own simulation top level.

Testbench	Main checks
`u_i_alu_tb.v`	LUI, AUIPC, immediate arithmetic, comparisons, logic, shifts.
`r_alu_tb.v`	Register arithmetic, bitwise operations, shifts, comparisons.
`loads_stores_tb.v`	Word, halfword, and byte loads/stores, including sign extension.
`branches_jumps_tb.v`	Taken and not-taken branches, backward loop, JAL, JALR.
`system_fence_tb.v`	FENCE, FENCE.TSO, PAUSE, ECALL, and EBREAK behavior.

Table 2: The five full-processor test groups.

We also tested the ALU, shifter, control units, memories, immediate generator, register file, decoder, multiplexers, and registers separately. The waveforms below show the control signals, PC changes, and

final data values. The testbenches also print PASS or FAIL messages for expected outputs.

5 Simulation Waveforms

5.1 U-type and I-type Instructions

These tests cover LUI, AUIPC, and immediate ALU operations. We checked the selected ALU operation, control signals, and resulting register values. For example, the tests expect 4096 from LUI and 40 from a left shift of 10 by two positions.

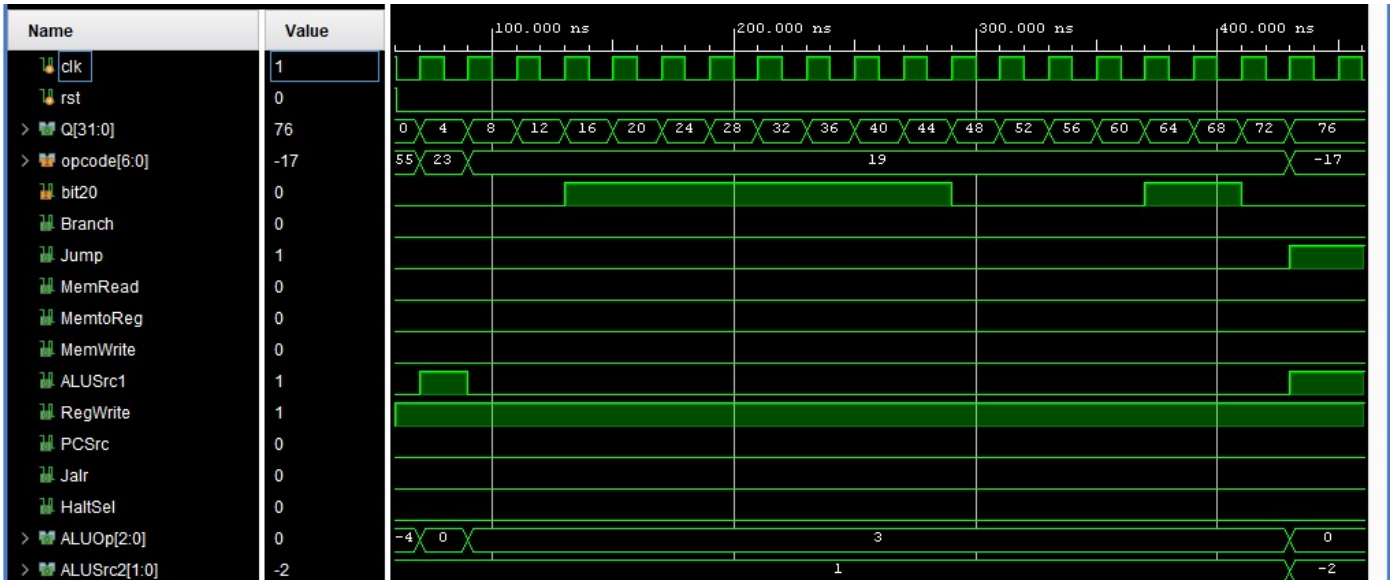


Figure 2: Control unit outputs during U-type and I-type instructions.
File: `immediate cu.jpeg`

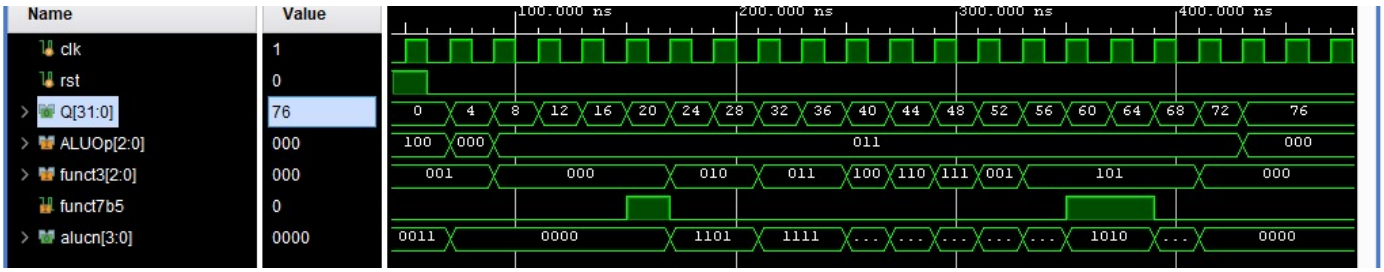


Figure 3: ALU control codes change for arithmetic, logic, and shift instructions.
File: `immediate alu cu.jpeg`

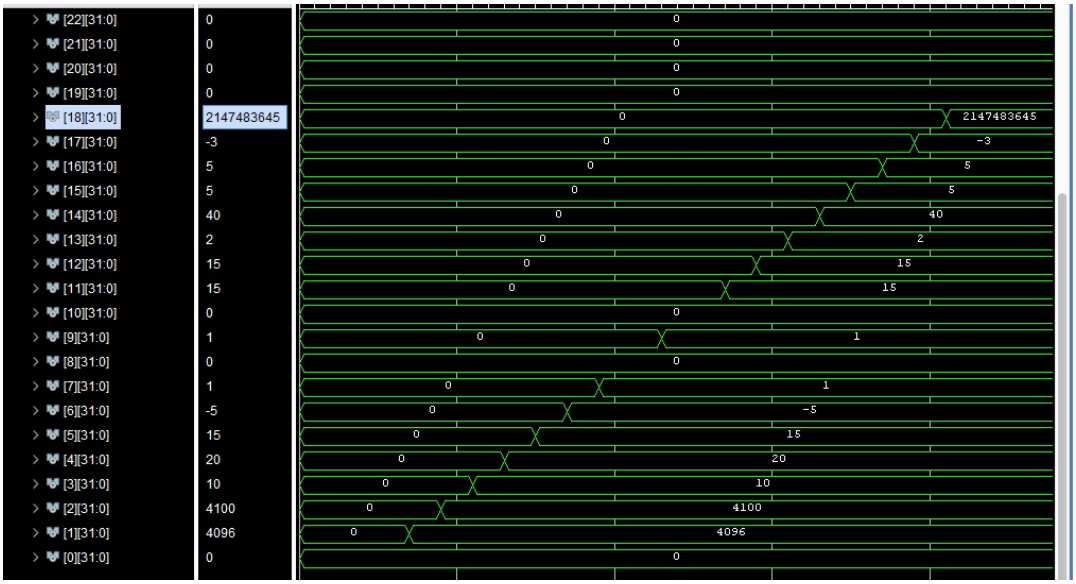


Figure 4: Register values after immediate and upper-immediate instructions.
File: `immediate alu registers.jpeg`

5.2 R-type Instructions

The R-type test checks ADD, SUB, shifts, comparisons, and bitwise operations. It includes negative values to check signed arithmetic. For example, subtracting 10 from 5 gives -5, and an arithmetic right shift of -8 by two gives -2.

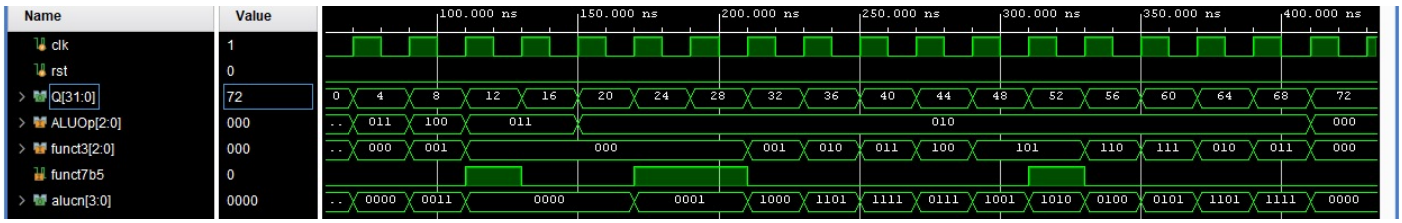


Figure 5: ALU control selects different R-type operations.
File: `r format alu cu.jpeg`

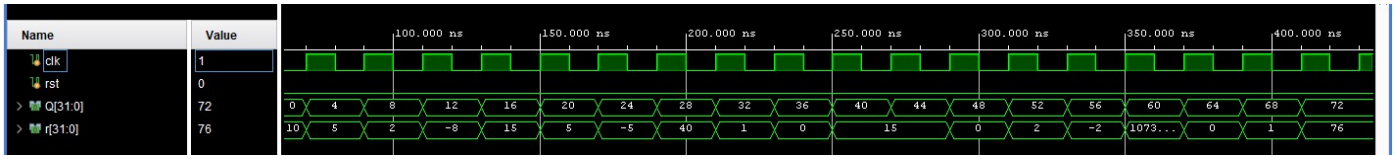


Figure 6: ALU results change as R-type instructions are executed.
File: `r format alu res.jpeg`

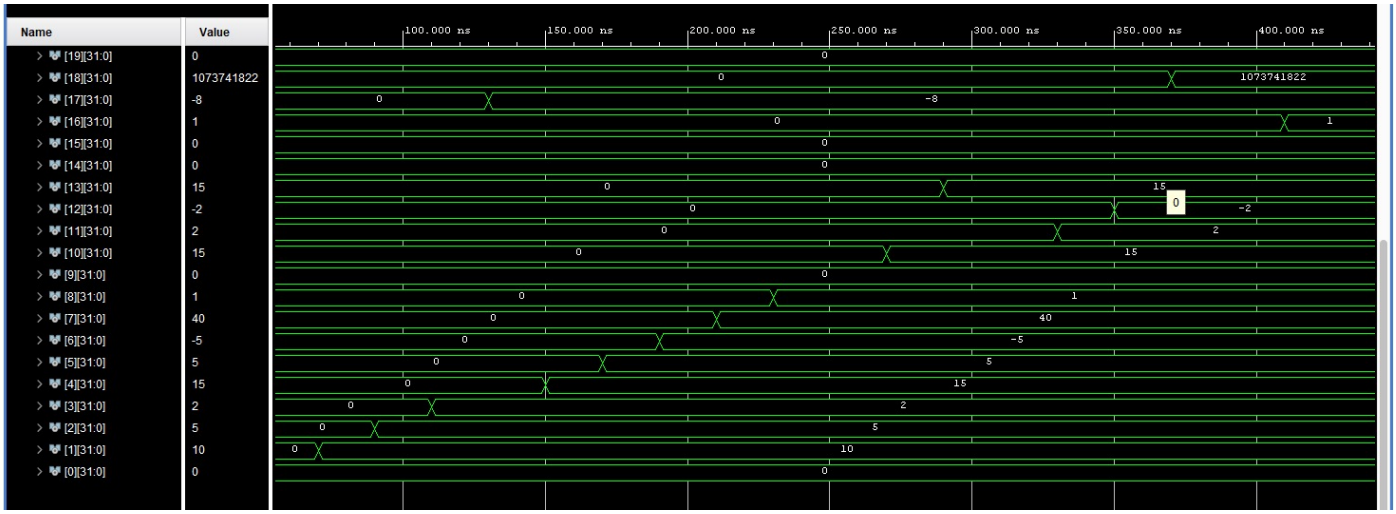


Figure 7: Final register values for the R-type program.
File: `r format registers.jpeg`

5.3 Loads and Stores

These tests check the memory read/write control signals and how bytes and halfwords are placed inside a 32-bit word. They also check signed and unsigned loads. For example, a stored byte with value 200 is read as -56 by LB and as 200 by LBU.

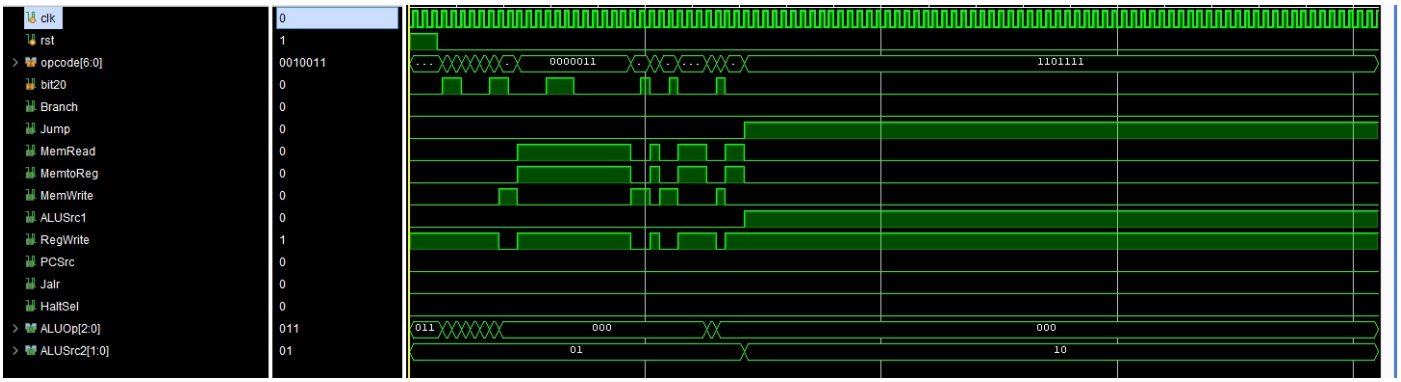


Figure 8: Load/store control signals, including memory read, memory write, and register write.
File: `load_stores cu signals.jpeg`

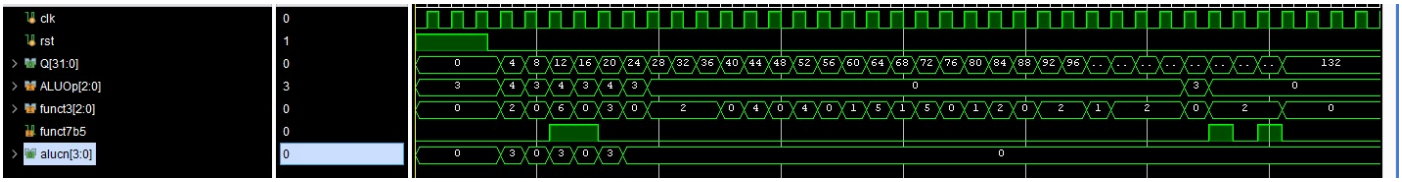


Figure 9: ALU control uses addition to calculate memory addresses.
File: `load_stores alu cu.jpeg`

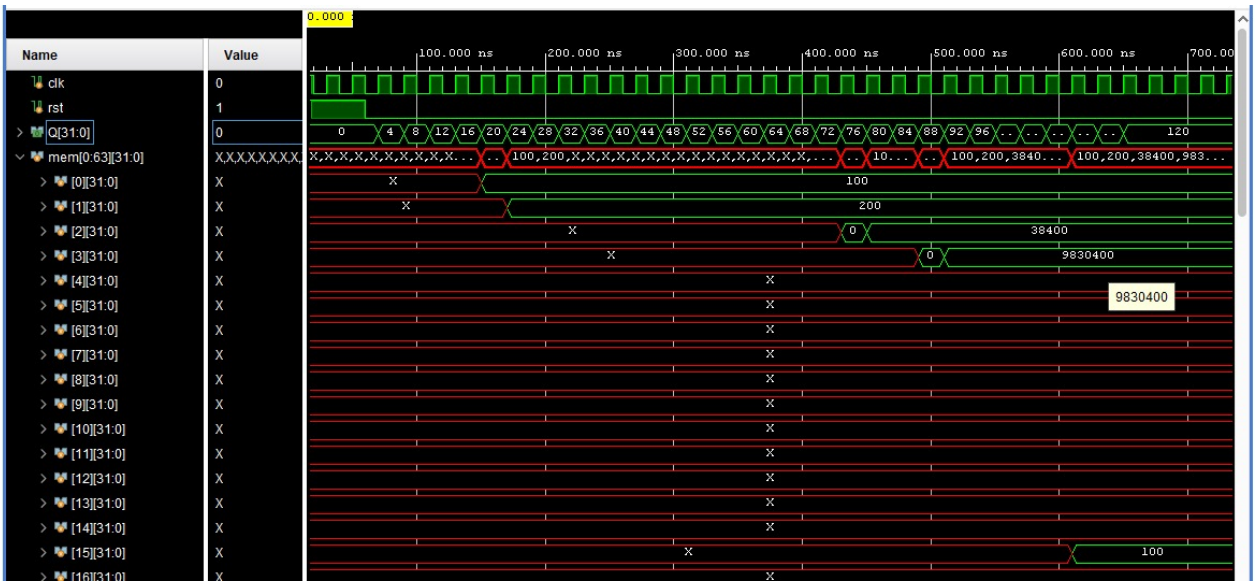


Figure 10: Data memory contents change after word, byte, and halfword stores.
File: `load_stores data memory.jpeg`

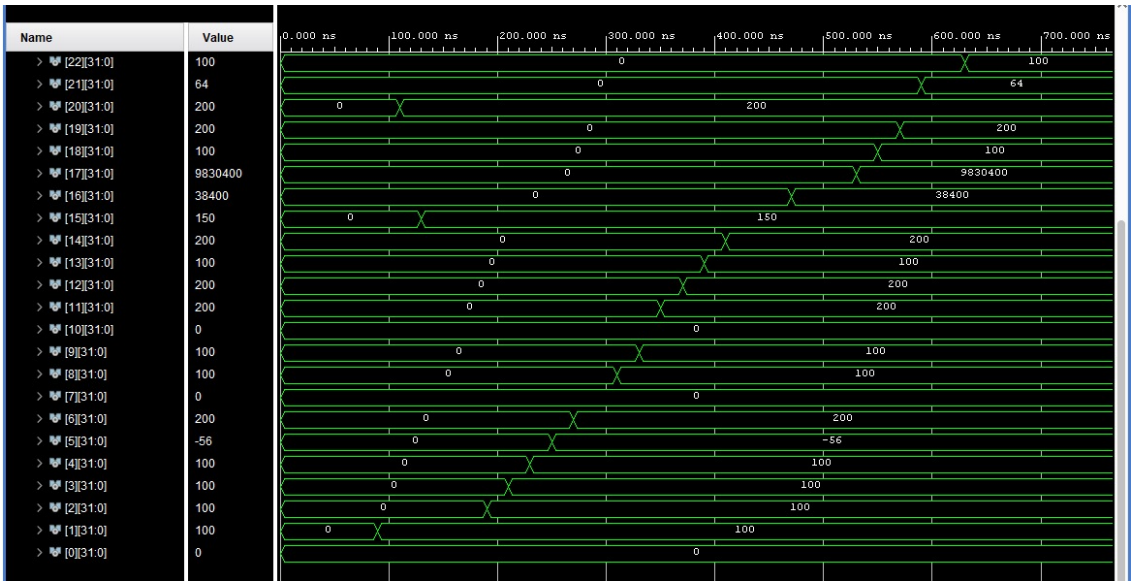


Figure 11: Registers contain the loaded words, bytes, and halfwords.
File: `load_stores registers 1.jpeg`

5.4 Branches and Jumps

These tests cover taken and not-taken branches, a backward loop, JAL, and JALR. We checked the PC changes and register values. In the test program, x27 should stay at zero for correctly taken branches, x28 should become 255 for correctly not-taken branches, and the loop counter x25 should reach three.

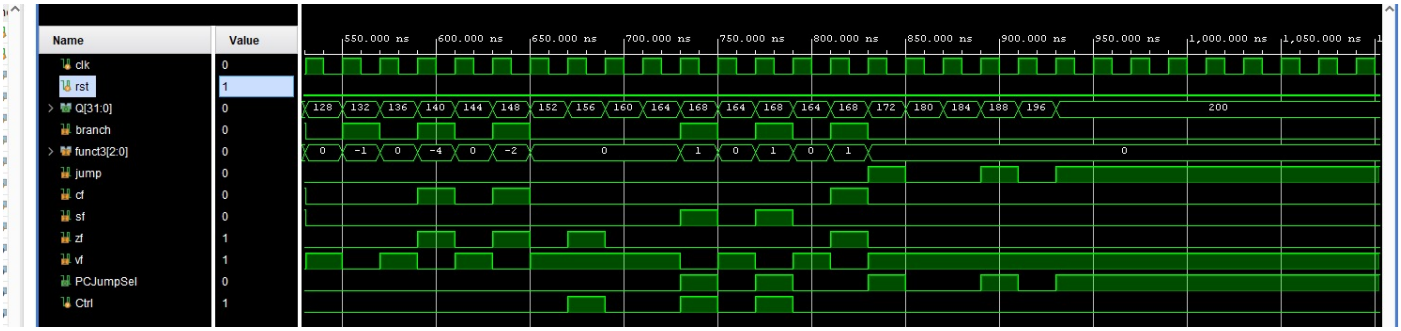


Figure 12: Branch control flags and PC selection during the first part of the test.
File: `branches_jumps branch cu 1.jpeg`

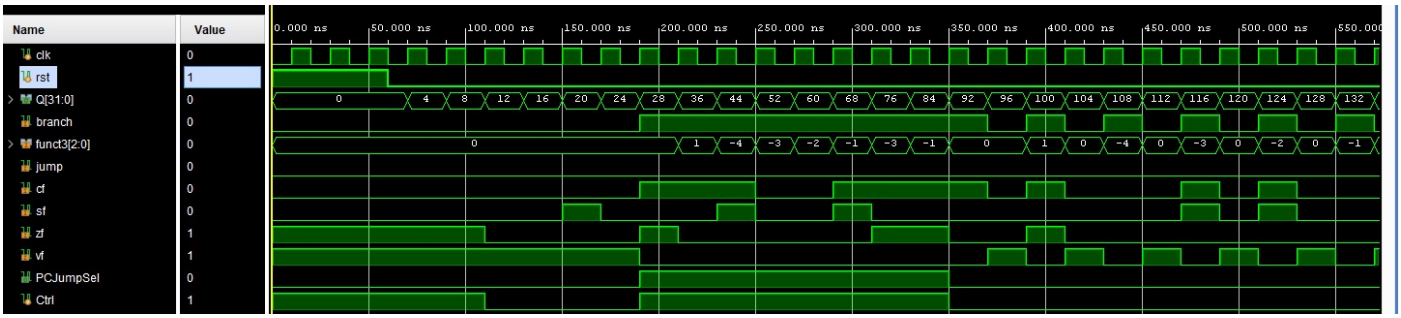


Figure 13: Branch decisions and jump selection later in the test.
File: `branches_jumps branch cu 2.jpeg`

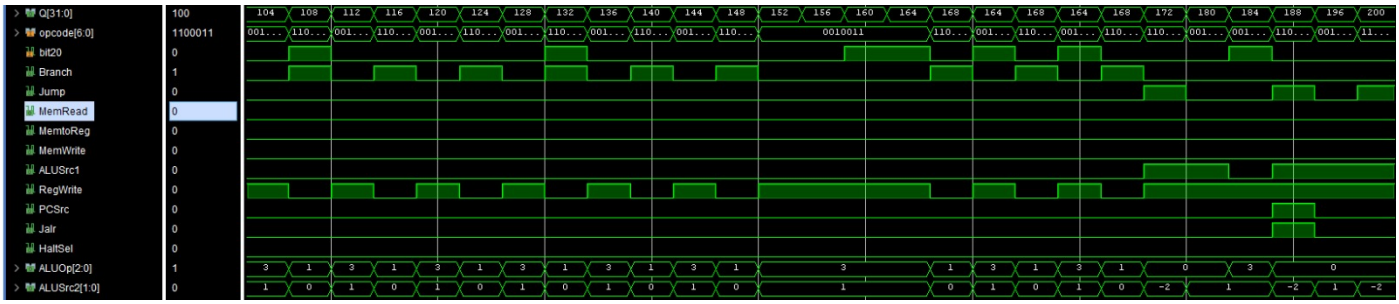


Figure 14: Main control signals while branches and jumps are executed.
File: `branches cu 2.jpeg`

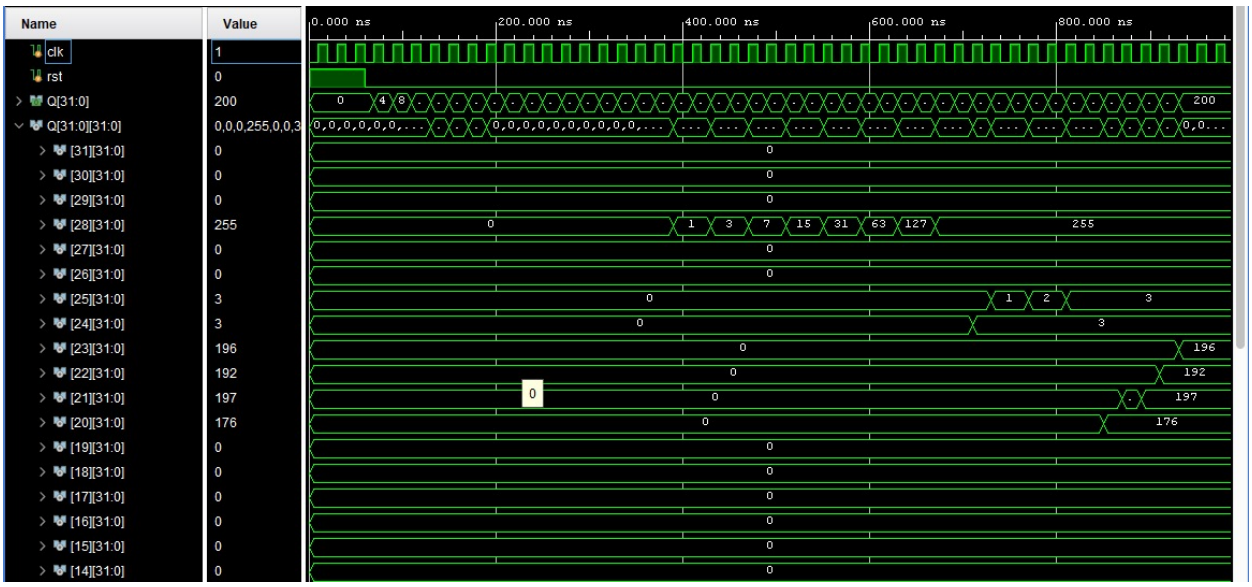


Figure 15: Register values during branch and jump testing, including x28 and x25.
File: `branches_jumps registers 1.jpeg`

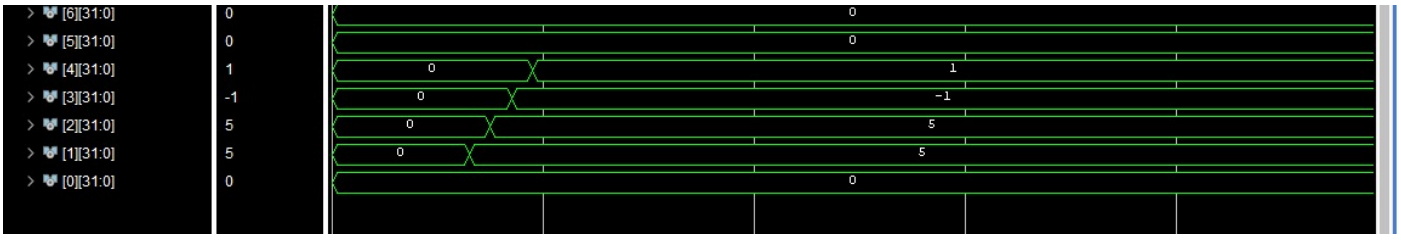


Figure 16: Initial comparison values used by the branch test.
File: `branches_jumps registers 2.jpeg`

5.5 System and Fence Instructions

For this project, FENCE, FENCE.TSO, PAUSE, and ECALL are treated as no-ops. EBREAK stops the PC. In the waveform, the PC reaches address 36 and then stays there while the halt signal is active.

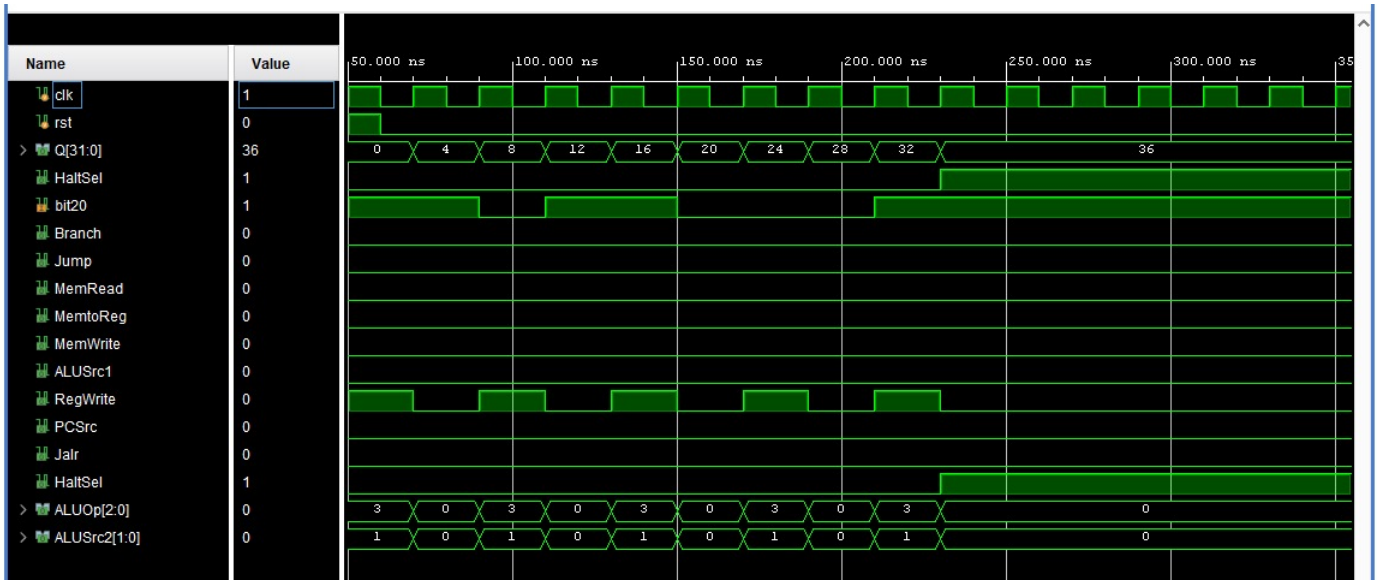


Figure 17: PC freezes at EBREAK while `HaltSel` stays high.
File: `fences.jpeg`

6 Challenges and Solutions

Branch decisions. Signed and unsigned comparisons need different conditions. We used the ALU flags (zero, sign, carry, and overflow) in the branch control unit to select the correct branch result.

Immediate values. RISC-V instruction formats place immediate bits in different positions. The immediate generator combines the bits for each format and sign-extends them when needed.

Small memory accesses. Byte and halfword instructions must access the correct part of a word. We used the lower address bits to choose the byte lane or halfword, then applied sign extension or zero extension for loads.

Jumps and halting. Jumps need both a new PC and a return address. We kept the target calculation separate from the PC+4 link value, cleared bit 0 for JALR, and used `HaltSel` to hold the PC on EBREAK.

7 Limitations and Conclusion

The memories are small and the testbenches load instructions directly. The design does not implement a full trap system: ECALL is treated like a no-op, and EBREAK only freezes the PC. The instruction memory uses the lower PC address bits, so programs must stay within its allocated range.

Overall, the project helped us understand how the datapath, control signals, ALU, memories, and register file work together. The test programs and waveforms show the expected behavior of the main instruction groups and give a practical way to check the design.